Bangladesh University of Business and Technology

Department of Computer Science and Engineering

CSE 463 : Software Design Pattern Lab

PROJECT REPORT

Semester: Summer 2026

Title: ParkSense — Smart Parking Management System

Submission Date: 26 August 2026

SUBMITTED BY

Full ID	Full Name	Intake/Section
22235103137	Md. Mahamudul Hasan	51/11

SUPERVISED BY

Talimul Bari Shreshtho
Lecturer, Department of CSE
Bangladesh University of Business and Technology (BUBT)

Abstract

ParkSense is a smart parking management system that digitises a multi-floor parking facility end to end: camera-simulated gated entry, a live bay-level occupancy map, tariffed ticketing with add-on services, monthly member processing, and an operator control room with LED-board and capacity-alert feeds. The system was built as a design-pattern laboratory: its architecture is organised around **16 Gang-of-Four design patterns**, each carrying a real responsibility in a real user flow rather than serving as decoration. The presentation tier is a dependency-free vanilla-JavaScript single-page application with a hand-written dark "control room" theme, hash routing and three-second live polling. The application tier is Java 17 on Spring Boot 3.4, deliberately kept framework-thin: the entire domain model is plain annotated-free Java wired in a single configuration class, with nine REST controllers exposing 36 endpoints through one facade. The data tier keeps in-memory repositories as the live source of truth and adds optional write-through persistence to MongoDB Atlas — a dedicated `parksense` database of seven collections whose state survives restarts, with a zero-setup in-memory fallback when no connection is configured. A deterministic seed boots a 132-bay, three-floor facility, roughly 35% occupied, plus a week of ticket history so that every report has data on first load. Authentication uses PBKDF2-hashed passwords with bearer tokens and three roles (ADMIN, OPERATOR, DEVELOPER), and a protection proxy guards emergency barrier overrides with an append-only audit trail. The platform delivers 13 functional modules, including the live parking map, the gate simulator, the add-on checkout lane, the five-strategy tariff engine and the pattern catalogue. A suite of **91 automated JUnit tests across 15 classes** verifies every pattern in isolation and the full entry-to-exit flow through the real HTTP surface.

Keywords: smart parking, design patterns, GoF, Java 17, Spring Boot, mediator, composite, strategy, REST API, single-page application, Bangladesh.

Acknowledgment

I express my sincere gratitude to my supervisor, **Talimul Bari Shreshtho**, Lecturer, Department of Computer Science and Engineering, Bangladesh University of Business and Technology, for his continuous guidance, valuable suggestions and encouragement throughout the development of this project. I am also thankful to the faculty members of the Department for their support, and to the open-source community — the Java, Spring Boot and Mermaid projects in particular — whose libraries and documentation made this work possible. Finally, I thank my family for their patience and encouragement during the entire CSE 463 cycle.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Problem Specification

Urban parking in cities such as Dhaka is operated largely by manual boom-gate lots: a guard notes a plate on paper, waves the car in, and settles the fare by memory at the exit. The consequences are familiar — queues at both barriers, lost-paper tickets disputed at the booth, no visibility of where free bays actually are, flat pricing that ignores commuters and events, no member handling, and no record of who overrode a barrier at 2 a.m. Commercial parking apps address fragments of the problem but are closed, rental products; none of them teaches — or even exposes — the architectural machinery that makes a gated facility safe and auditable. For a software design laboratory, the parking domain is therefore doubly attractive: it is a real problem worth solving, and it is an unusually dense supplier of the classic design problems — object identity, tree-structured aggregates, rule chains, state machines, hardware integration and coordinated reactions to shared events. These gaps motivated ParkSense: a complete, self-contained smart-parking system whose architecture is the exhibit, built from 16 GoF design patterns that each carry real load in a real flow.

1.2 Objectives

The principal objectives of the project were:

1. To build a smart parking management system with a vanilla-JavaScript single-page front end, a Java 17 / Spring Boot 3.4 application tier and a fully in-memory data tier requiring zero installation.
2. To automate gated vehicle entry through an ANPR-simulated camera read, a rule-based entry chain and sensor-confirmed bay allocation.
3. To price every stay through a selectable tariff engine with five strategies, vehicle-class factors, grace windows and stacking add-on services.
4. To allocate bays by vehicle class and accessibility badge, with specialised-bay preference and charger-fallback policies.
5. To secure the platform with PBKDF2 password hashing, bearer-token sessions and a three-role model (ADMIN, OPERATOR, DEVELOPER) enforced at the HTTP edge and at the object boundary.
6. To keep the architecture modular, pattern-explicit and testable — the entire domain is plain Java with no framework annotations.
7. To give operators a live control room: bay-level map, LED boards, capacity alerts, event feed and an append-only audit trail.
8. To verify every design pattern with focused automated tests — 91 JUnit tests across 15 classes — including a full-stack entry-to-exit flow test through the real HTTP surface.

1.3 Scope

The system in scope covers staff authentication and role-based access; the parking facility model (lot, floors, zones, five bay types, 132 bays); simulated ANPR entry with a six-rule authorization chain; ticketing with a complete state machine; checkout with cash/card/mobile settlement and change computation; add-on services (car wash, valet, EV top-up, lost-ticket penalty); monthly member passes with an express exit lane; tariff administration with builder-validated plans and live activation; reporting (revenue, occupancy, utilisation, peak hours, add-on uptake) with CSV export; live boards, feeds and alerts; an audit trail; and a self-documenting design-pattern catalogue. Out of scope — and discussed under future work — are real payment-gateway integration, physical camera and sensor hardware, native mobile applications, advance bay reservation for drivers, multi-site federation, and machine-learning demand prediction.

1.4 Flow of the Project

Development was organized into six phases:

Phase 1 — Planning & Design (Weeks 1–2): Requirements gathering, use-case modelling, lot-tree and store design, pattern responsibility mapping, architecture planning and UI wireframing.

Phase 2 — Domain Core (Weeks 3–5): Lot composite, occupancy ledger and event publisher, vehicle and plate registries, tariff strategies with builder, ticket state machine and add-on decorators, in-memory stores, seeded demo data.

Phase 3 — Hardware & Coordination (Weeks 6–7): Vendor SDK stand-ins and adapters, hardware factories, gates with command queues, the protection proxy, the control-room mediator, entry chain and exit-lane template processors.

Phase 4 — HTTP & Front-End (Weeks 8–10): Nine REST controllers behind the operations facade, authentication filter, the single-page control room with all nine views and live polling.

Phase 5 — Testing & QA (Weeks 11–13): 15 JUnit test classes (91 tests), headless-browser QA of every page and flow, defect fixing and security hardening of the proxy and auth paths.

Phase 6 — Documentation (Weeks 14–15): README, this report, diagram preparation and presentation preparation.

1.5 Organization of the Project Report

The remainder of this report is organized into six chapters. **Chapter 2** surveys existing systems and the supporting literature. **Chapter 3** presents the system analysis and design — the technology stack, the development model, the system architecture, the use-case, context and data-flow diagrams, the store schema, the sequence, activity and state diagrams, and the core algorithms. **Chapter 4** describes the implementation

of the front end and back end, the 16 design patterns as realised in the code, and enumerates the functional modules. **Chapter 5** is a user manual covering system requirements and interface walkthroughs. **Chapter 6** concludes with a discussion of limitations and future work, followed by the references and appendices.

Chapter 2

Background and Related Works

2.1 Existing System Analysis

Several commercial products and the ubiquitous manual lot were studied before ParkSense was designed. Table 2.1 summarises the strengths and weaknesses of the most relevant alternatives.

Table 2.1: Comparative analysis of existing platforms.

Platform	**Strengths**	**Weaknesses**
Manual boom-gate lots (local baseline)	No technology cost; flexible human judgement	Paper tickets, fare disputes, no occupancy visibility, no audit trail, no dynamic pricing
ParkMobile	On-street payments and reservations at scale; strong mobile app	Street-parking focus, no gated-facility bay map, closed architecture
SpotHero	Large garage inventory marketplace, pre-booking, clear pricing	Reservation-centric; nothing for walk-up gated operation or add-on services
JustPark	Drill-down search, driveway + car-park supply, upfront pricing	Marketplace model; no lane hardware simulation, no operator control room
Passport Labs	Municipal end-to-end permitting, enforcement and payments	City-scale focus; opaque to a single lot operator, no educational transparency

The recurring gaps identified were:

- No bay-level live occupancy map for a walk-up gated facility — drivers circle, operators guess.
- Rule-based entry authorisation (blacklist, duplicate stay, vehicle class fit) with driver-readable refusal reasons is absent from consumer products.
- Pricing is rigid: early-bird, event surge, daily caps and add-on services rarely coexist in one configurable engine.
- Manual barrier overrides, where they exist at all, are not guarded by role checks nor written to a tamper-evident audit trail.
- None of the studied systems documents — or even exposes — its internal architecture, making them unusable as a design laboratory.

ParkSense addresses every one of these gaps: it couples a live bay map and a rule-chained gated entry with a five-strategy tariff engine, add-on services, member express lanes, a role-guarded audited barrier override, and a built-in catalogue that documents the 16 design patterns the system itself is built on.

2.2 Supporting Literatures

The work draws on several bodies of knowledge — the classical design-pattern catalogue, smart-city and intelligent-transport literature, modern web architecture patterns, authentication practice, and the specific front-end and back-end technologies employed.

Design-pattern foundations. The architecture is grounded in the Gang-of-Four catalogue [1] and its modern expositions [3, 2]: creational patterns build valid objects (ledger singleton, hardware factory, tariff builder), structural patterns compose them (lot composite, fee decorators, hardware adapters, guard proxy, operations facade), and behavioural patterns coordinate them (control-room mediator, occupancy observer, tariff strategies, gate commands, exit template, entry chain, ticket state machine, report visitors).

Intelligent transport systems. Smart-parking surveys [5] consistently identify guided bay search, automated gated entry and demand responsive pricing as the three highest-impact services; ParkSense implements a self-contained laboratory version of each.

Architecture. The system follows a classical three-tier shape: a vanilla-JavaScript single-page presentation layer, a Spring Boot application layer with one facade in front of nine thin controllers, and a data tier of in-memory repositories mirrored by a write-behind MongoDB bridge — deliberately annotation-free so that every pattern remains ordinary, testable Java [7, 4, 10].

Authentication and authorization. Passwords are stored only as PBKDF2-HMAC-SHA256 hashes with per-user salts [14]; sessions are opaque bearer tokens resolved by a servlet filter that publishes the caller's role to a thread-local context, which both the HTTP edge and the gate-control proxy consult.

Front-end stack. The SPA uses no framework by design: hash routing, a three-second polling store and a `fetch` wrapper are each small enough to read in one sitting, which suits a teaching artefact and keeps the delivered bundle dependency-free [17].

Back-end stack. Java 17's sealed switch expressions and records shrink the pattern classes; Spring Boot 3.4 contributes only the embedded web server and JSON binding [7, 9]; Maven wraps the build so the project runs with a single command on any JDK 17+ machine [12].

2.3 Problem Statement and Research Gap

The specific problem this project solves can be stated as follows: *a single parking-facility operator lacks an integrated, self-hosted system that simultaneously provides (a) bay-level live occupancy guidance, (b) rule-based automated gated entry with driver-readable refusals, (c) a configurable multi-strategy tariff engine with stacking add-on*

services, (d) member express processing, and (e) role-guarded, fully audited manual control — in an architecture whose design decisions are explicit, documented and testable. Existing solutions each satisfy at most one or two of these requirements. ParkSense is the integrated response to that research gap.

Chapter 3

System Analysis and Design

3.1 Technology and Tools

3.1.1 Front-End Stack

Table 3.1: Front-end technologies and their roles.

Technology	Version	Role
HTML5	—	Document shell of the single-page application
CSS3 (hand-written)	—	Dark "control room" design system: panels, LED boards, slot grid, barrier animation
JavaScript (ES2020)	—	Nine views, hash router, live store, API client — zero dependencies
Fetch API	standard	Bearer-token HTTP client with unified error handling
Hash routing (custom)	—	Nine routes guarded by session state
CSS Grid	—	Bay grid, KPI tiles, gate panels, chart bars

3.1.2 Back-End Stack

Table 3.2: Back-end technologies and their roles.

Technology	Version	Role
Java	17	Domain language for all 16 patterns; records, switch expressions
Spring Boot	3.4.0	Embedded Tomcat 10.1, JSON binding, test harness only
spring-boot-starter-web	3.4.0	Web tier (the only starter dependency)
mongodb-driver-sync	5.2.1	Optional write-behind persistence to MongoDB Atlas
Jackson	2.17	DTO serialisation via Java records
JUnit 5 + MockMvc	5.10/6.2	91 tests across 15 classes
Maven (wrapper included)	3.9.x	One-command build, test and run

3.1.3 Data Tier

The live source of truth is a set of seven in-memory repositories and registries (plain thread-safe Java collections) holding the document models listed in Table 3.3. A write-behind `MongoSync` bridge mirrors every change into a dedicated `parksense` MongoDB Atlas database (collections: `tickets`, `members`, `tariffs`, `users`, `audit_log`, `lot_state`, `counters`) so that state — open stays, settled fares, members, tariffs, the audit trail, even out-of-service bays and the ticket-number sequence — survives restarts. On boot the system loads the database when it holds a previous state and seeds the deterministic demo only otherwise; with no reachable connection it logs a

friendly notice and runs in zero-setup in-memory mode exactly as before. An authenticated `reset` endpoint drops the collections and replays the seed at any time.

Table 3.3: In-memory stores and their document models.

Model	Purpose
TicketStore / Ticket	Parking sessions with state machine, fee chain and payments
MemberStore / Member	Monthly pass holders and their plates
TariffStore / TariffPlan	Five pricing plans, at most one active per kind
UserStore / User	Staff accounts with PBKDF2 hashes and roles
PlateRegistry	Blacklist and the plates-inside set
AuditTrail / AuditEntry	Append-only log of every command, denial and settlement
OccupancyLedger + lot tree	Live slot states: ParkingLot, Floor, Zone, Slot

3.1.4 Tools Used

Table 3.4: Development and testing tools.

Tool	Purpose
Visual Studio Code	Java and JavaScript editing with Spring Boot extensions
Apache Maven	Dependency management and reproducible builds (wrapper bundled)
Postman	REST endpoint probing during development
Chrome DevTools	SPA debugging, network inspection
Headless-browser QA script	Automated walkthrough of all nine views with console-error capture
mermaid.live	Rendering of every diagram in this report

3.2 Development Model

The project followed an incremental-agile model: each phase ended with a runnable, demonstrable system (the domain compiled and unit-tested before any HTTP existed; the API was smoke-tested with Postman before the SPA wired to it), and the pattern catalogue was kept truthful by regenerating documentation from the code. Table 3.5 summarises the phases and Figure 3.1 the timeline.

Table 3.5: Sprint plan and deliverables.

Phase	Duration	Deliverable
Planning & Design	Weeks 1–2	Use cases, lot-tree and store models, pattern responsibility map
Domain Core	Weeks 3–5	Composite, singleton ledger, strategies, builder, state machine, decorators, seeded stores
Hardware & Coordination	Weeks 6–7	Adapters, factories, command queues, proxy, mediator, chain, exit templates
HTTP & Front-End	Weeks 8–10	Nine controllers, auth filter, nine SPA views with live polling
Testing & QA	Weeks 11–13	91 JUnit tests, headless page QA, security fixes
Deployment & Docs	Weeks 14–15	README, report, diagrams, presentation

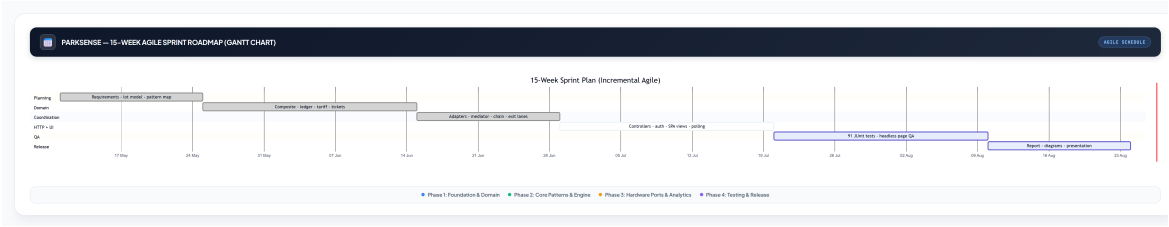

Figure 3.1: Sprint plan Gantt chart.

3.3 System Architecture

Figure 3.2 shows the three tiers. The browser SPA talks HTTPS + bearer token to a thin REST edge (nine controllers, one filter) that delegates every operation to the `OperationsFacade`. Behind the facade the plain-Java domain cooperates through the 16 patterns: the mediator coordinates the lane colleagues, the singleton ledger guards slot truth, the command queues drive lane hardware through adapter-wrapped vendor ports, and the event publisher fans occupancy changes out to boards, feeds and alerts. There are no external services — even the vendor hardware is represented by in-process SDK stand-ins, so the whole facility runs from one command.

3.4 Use-Case Analysis

Four actors interact with the system; the ADMIN inherits every OPERATOR use case. Their principal use cases are summarised in Table 3.6 and Figure 3.3 shows the use-case diagram.

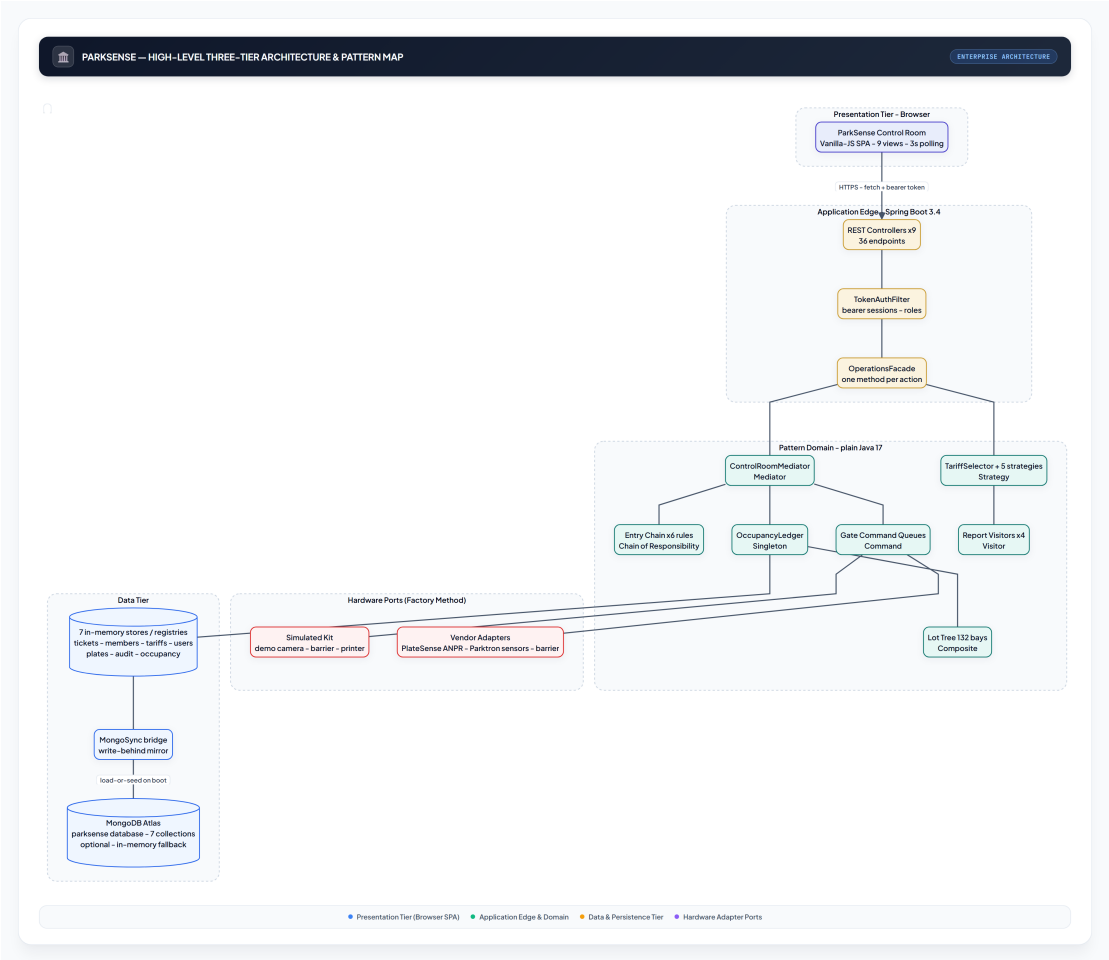

Figure 3.2: High-level three-tier system architecture of ParkSense.

Table 3.6: Use-case summary by actor.

Actor	Representative use cases
Operator	Sign in; drive vehicle entry (camera read); process exits incl. lost tickets; settle fares with change; toggle add-ons; browse live map, boards and events; undo gate commands
Administrator	Inherits all Operator use cases; force-open/close barriers; void tickets; manage members; build and activate tariffs; run reports with CSV export; read the audit trail; reset/reseed the system
Member / Driver (external)	Enter by recognised plate; park in an assigned bay; settle at kiosk or express lane; receive grace window after payment
Sensor network & vendor hardware (external)	Report slot presence; deliver ANPR frames; execute barrier motor commands

3.5 Context-Level Diagram (DFD-0)

The context diagram (Figure 3.4) shows the system as a single process exchanging flows with its external entities: the operator staff, the administrator, member drivers, and the vendor hardware boundary.

3.6 Data-Flow Diagram (DFD-1)

The first-level decomposition (Figure 3.5) breaks the system into six cooperating subsystems — authentication, gated entry, occupancy management, checkout and exit, tariff administration, and reporting — all of which read from and write to the shared in-memory stores.

3.7 Store Schema

The entity relationships among the in-memory documents are shown in Figure 3.6. Tickets reference their bay, gate and tariff plan; members own plates; the lot tree owns bays whose live state is guarded by the ledger. Selected field-level detail is given in Table 3.7.

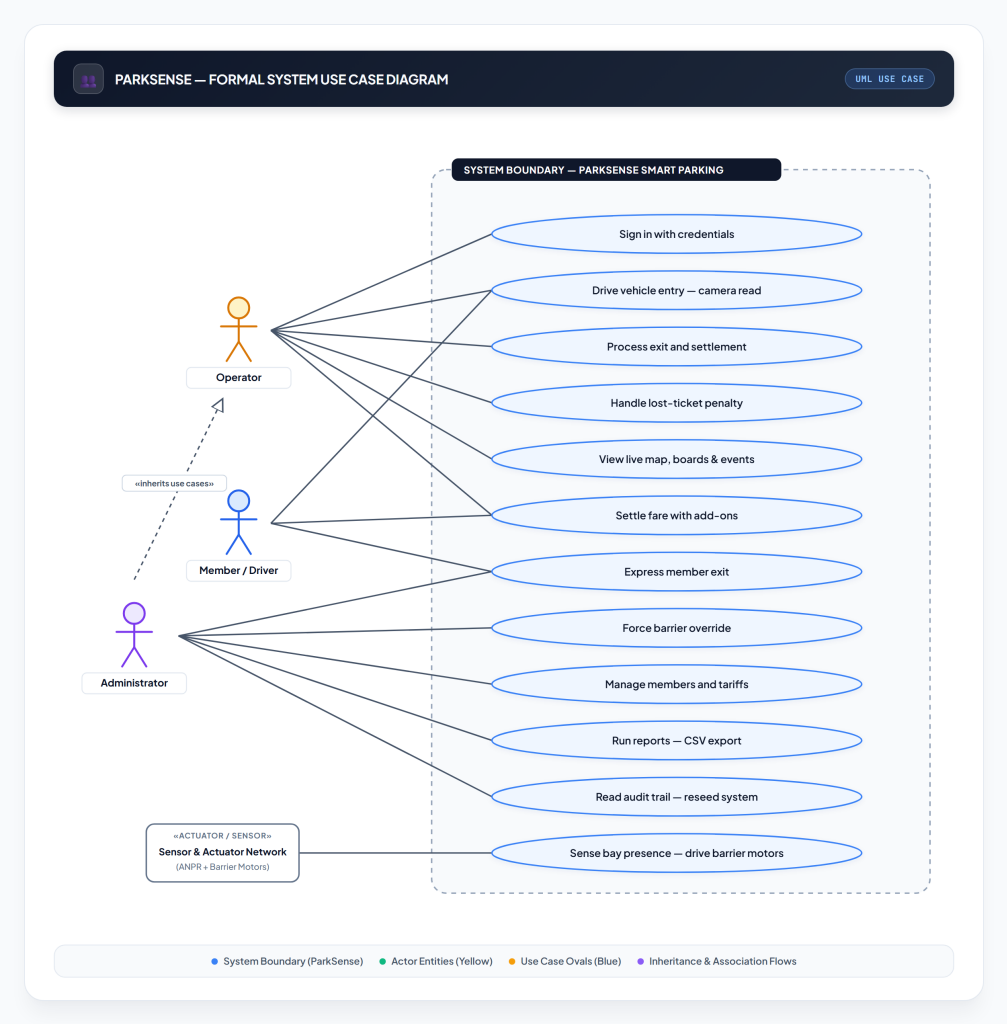

Figure 3.3: System Use Case Diagram.

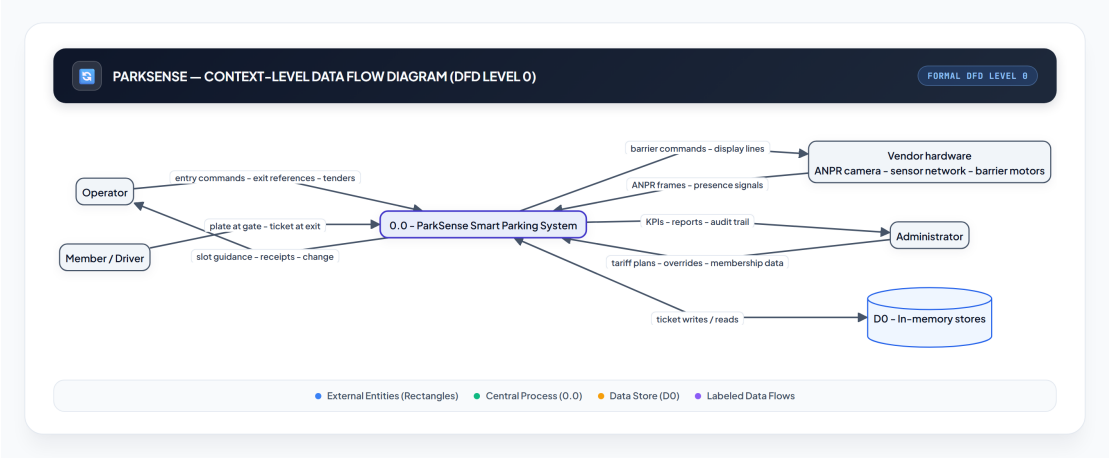

Figure 3.4: Context-level diagram (DFD-0).

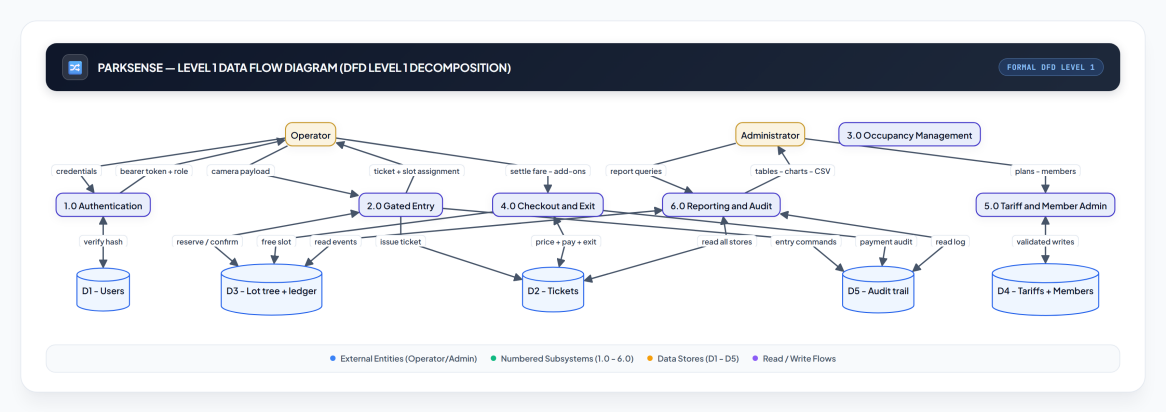

Figure 3.5: First-level data-flow diagram (DFD-1).

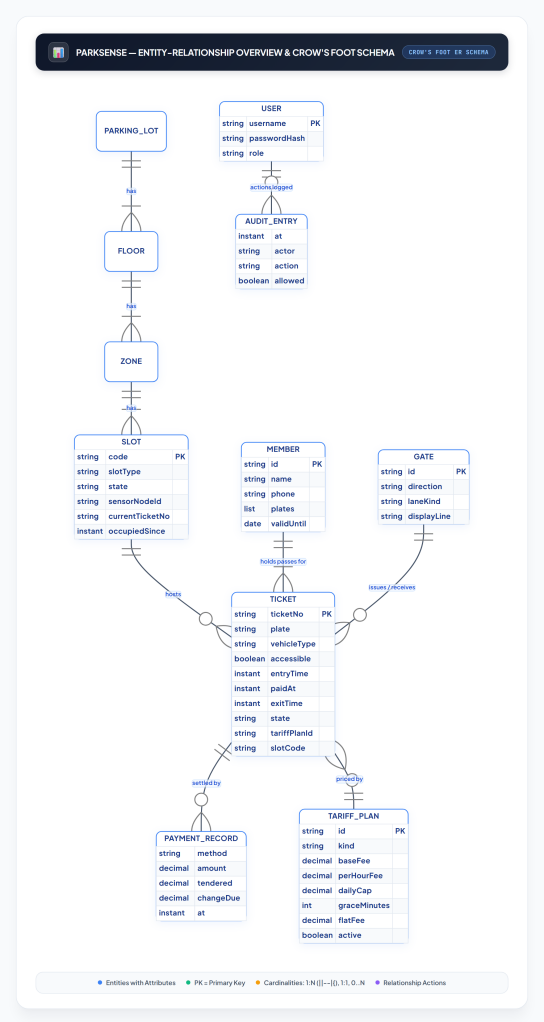

Figure 3.6: Entity-relationship overview of the ParkSense stores.

Table 3.7: Selected schema field detail.

Model	Key field	Notes
Ticket	`state`	ISSUED, ACTIVE, PAID, EXITED, LOST, VOID — transitions guarded by the State pattern; illegal calls throw
Ticket	`tariffPlanId`	The plan selected at settlement; recorded with the selector's reason for audit
Slot	`state`	FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE — only the ledger may write it
TariffPlan	`kind`	One of five kinds; at most one active plan per kind, enforced by the store
Member	`plates`	One or more plates; pass validity checked against the simulated clock
User	`passwordHash`	PBKDF2-HMAC-SHA256, 120k iterations, per-user salt
AuditEntry	`allowed`	Denials and approvals share one append-only log

3.8 Sequence Diagrams

Two flows carry the system. Figure 3.7 shows a vehicle entry — the camera adapter, the rule chain, the mediator's allocation, the command queue and the observer fan-out in one pass. Figure 3.8 shows the exit and payment flow — the template pipeline, the tariff strategy, the fee decorators and the state machine settling the ticket before the barrier opens.

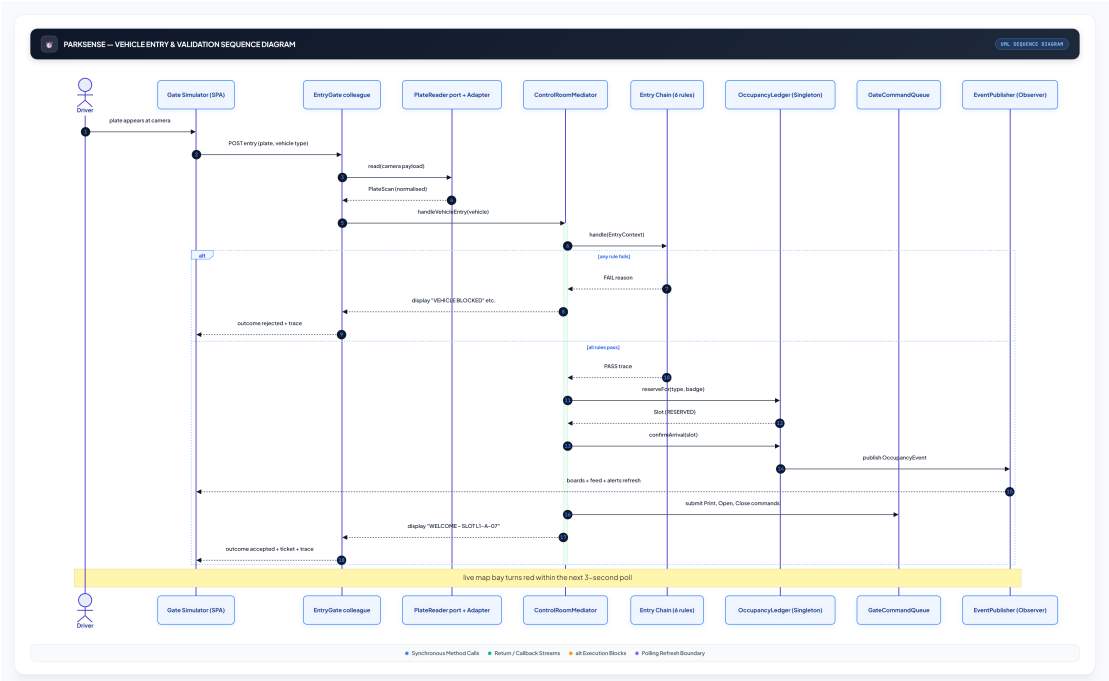

Figure 3.7: Sequence diagram for vehicle entry.

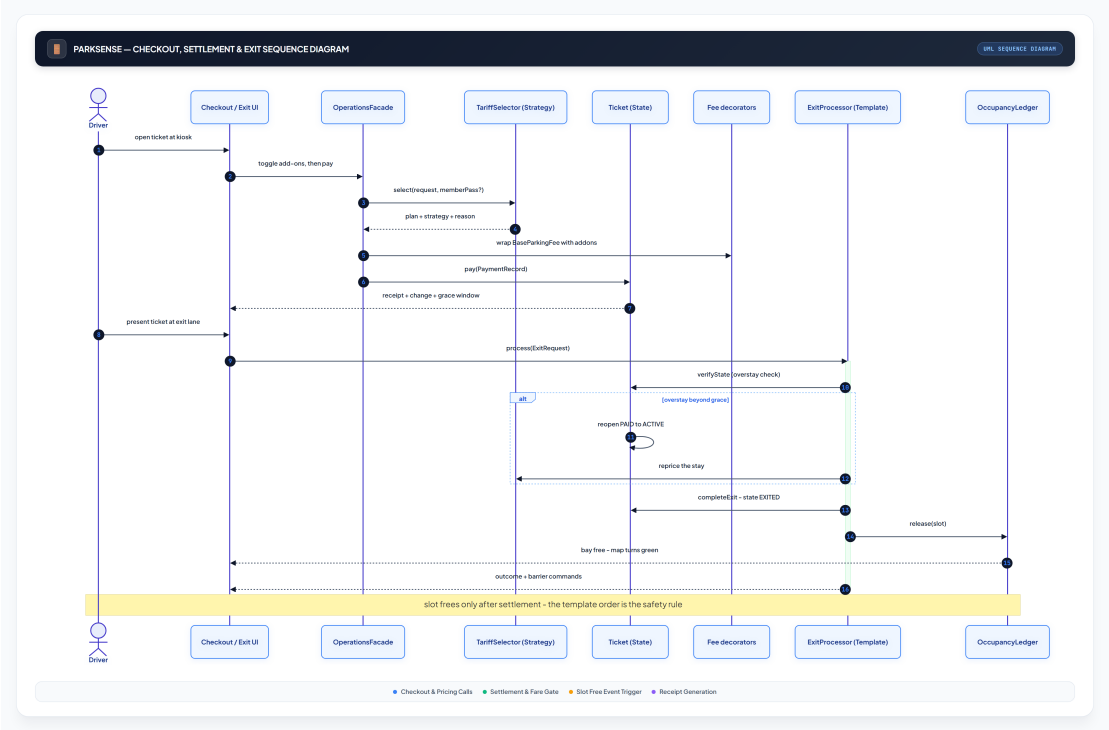

Figure 3.8: Sequence diagram for checkout, payment and exit.

3.9 Activity Diagram

The activity diagram (Figure 3.9) details the complete visitor journey: arrival, rule checks, bay allocation, parking, optional add-on services, settlement, the grace window and exit — including the lost-ticket path and the overstay reopening.

3.10 State Diagrams

Two lifecycles matter. The ticket state machine (Figure 3.10) is the GoF State pattern itself; the bay lifecycle (Figure 3.11) is guarded by the ledger.

3.11 Algorithms and Flowcharts

3.11.1 Specialised-Bay Allocation

The entry flow's core decision is which bay a vehicle receives (Figure 3.12). Inputs are the vehicle class and accessibility badge; the policy prefers specialised bays for the vehicles that need them (a motorcycle never takes a standard bay, an EV gets a charger before a plain bay, a badge holder gets an accessible bay first) so that general bays remain for general cars. The chain's availability rule consults the same preference order, so a refusal reason and the allocator can never disagree.

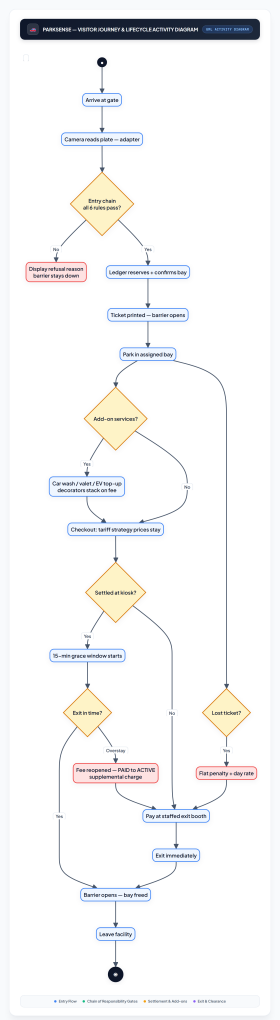

Figure 3.9: Activity diagram for the visitor journey.

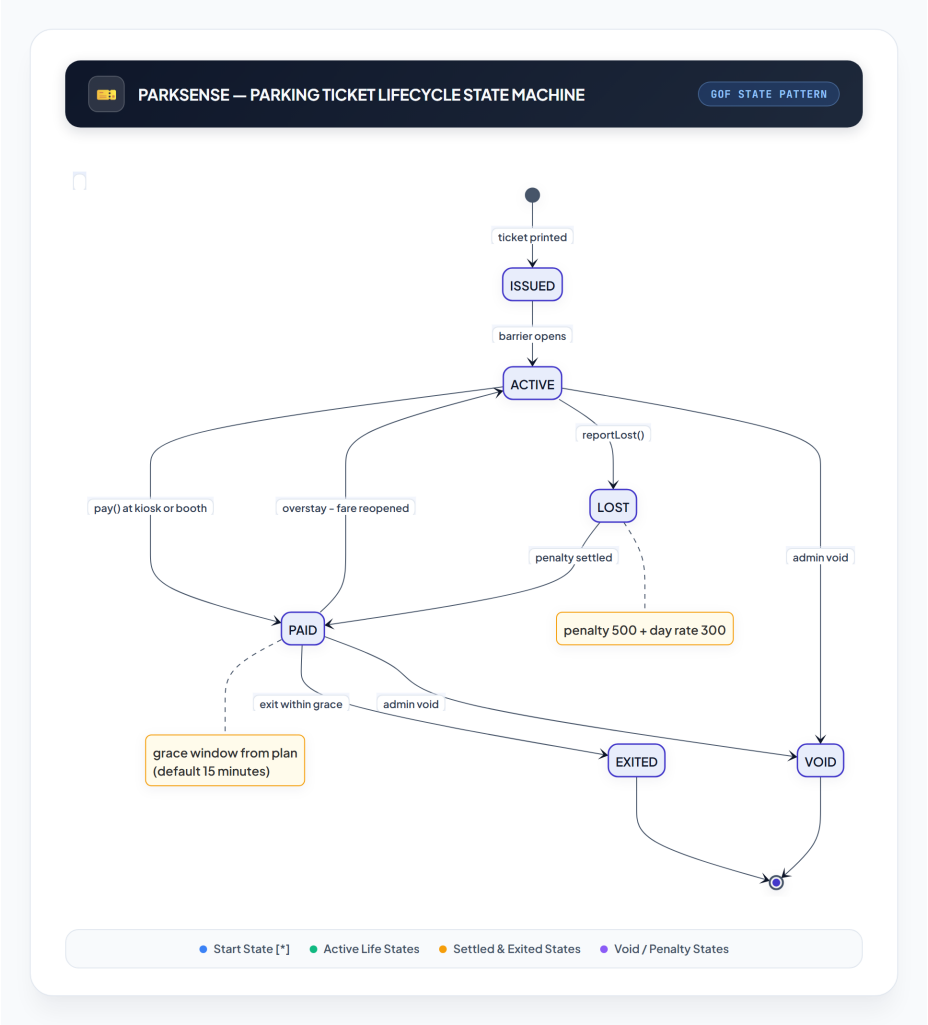

Figure 3.10: Ticket lifecycle state diagram (State pattern).

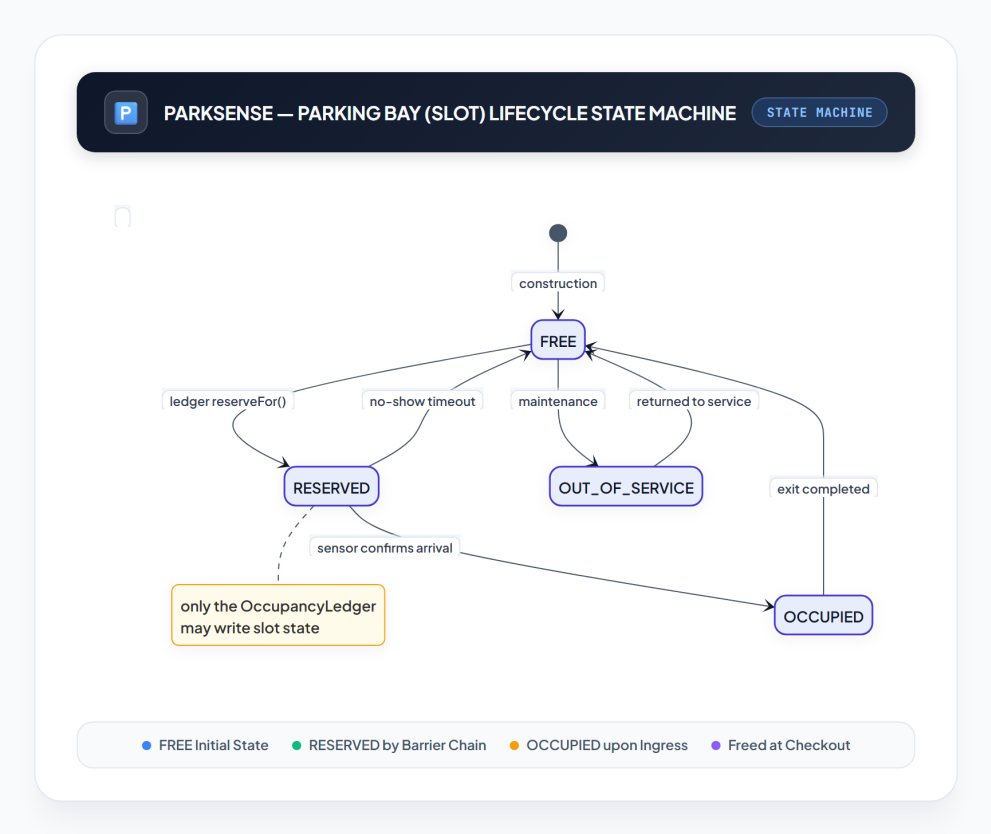

Figure 3.11: Bay (slot) lifecycle state diagram.

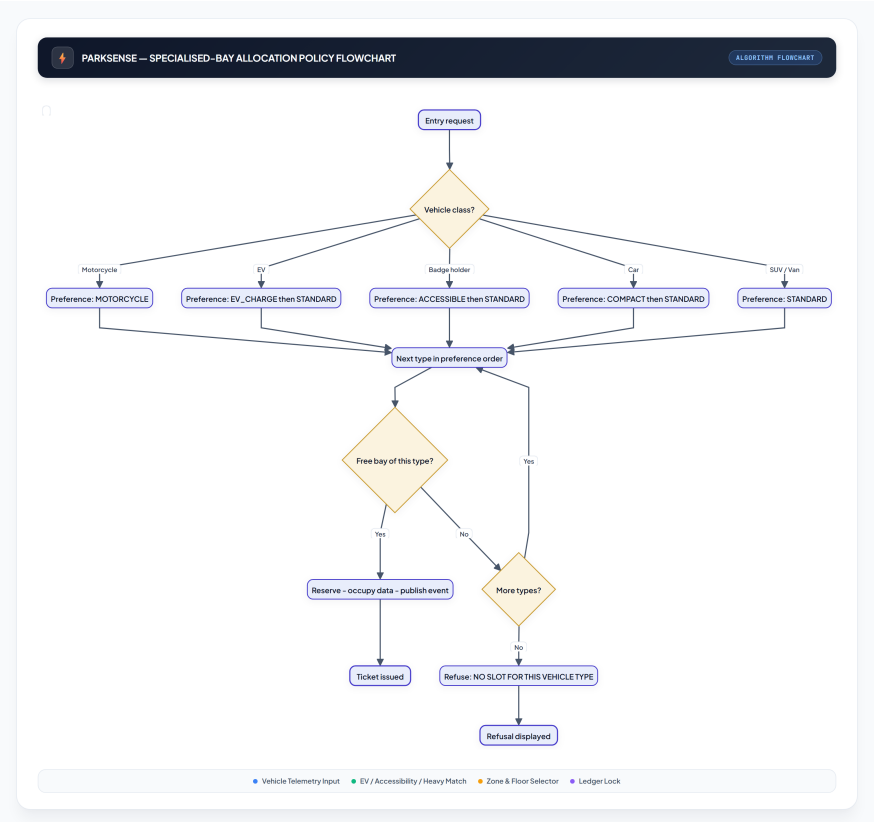

Figure 3.12: Specialised-bay allocation flowchart.

3.11.2 Authentication Flow

Staff authenticate once per session (Figure 3.13); every subsequent call carries the opaque bearer token, which the filter resolves to a thread-local user that both the endpoints and the gate-control proxy consult. Operators are refused at the proxy, not merely in the UI, and every attempt is audited.

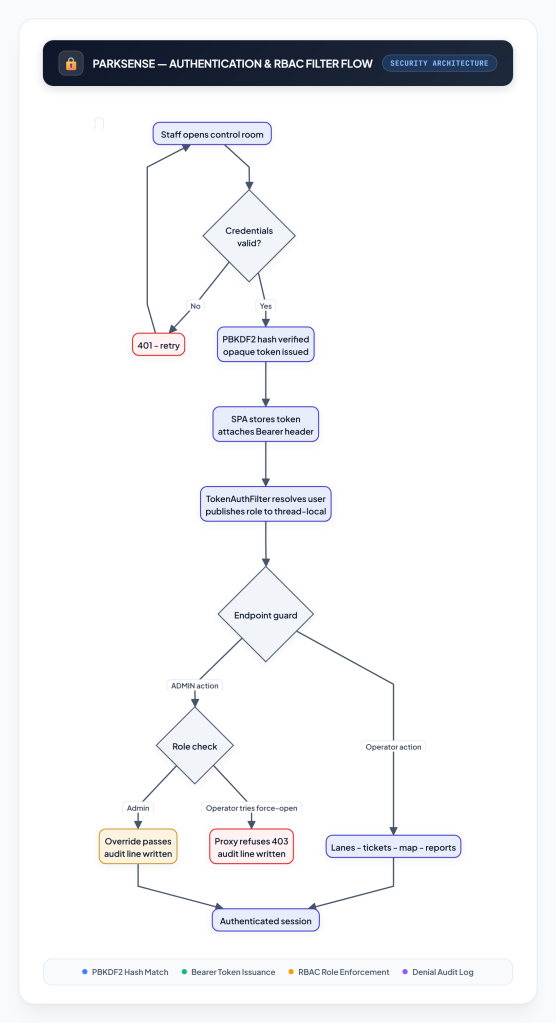

Figure 3.13: Authentication and role-guard flow.

3.11.3 Tariff Selection

At settlement the selector walks a fixed priority ladder (Figure 3.14): a valid member pass outranks everything, an active event-surge plan comes next, then the early-bird window test, then the capped-hourly default, then plain hourly. Because activation is operator-controlled, pricing changes are data changes — no code moves.

3.12 Facility Composition and Seeded Analytics

The seeded facility mixes five bay types across its 132 bays (Figure 3.15); the rooftop level is deliberately motorcycle- and EV-heavy. A week of seeded history gives the

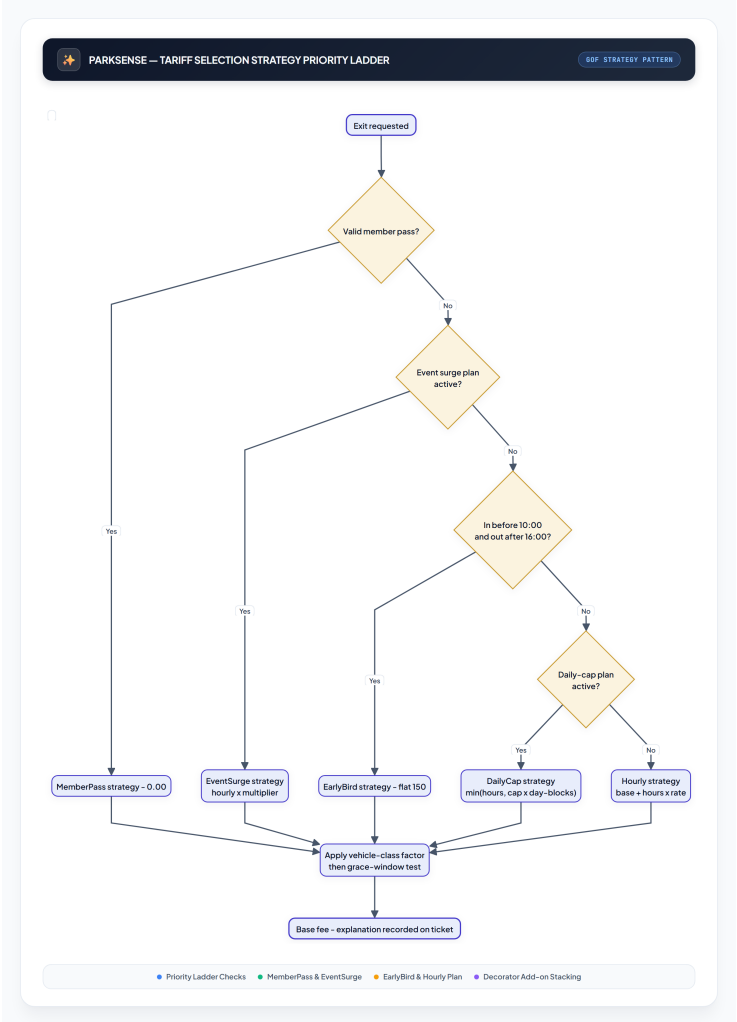

Figure 3.14: Tariff selection strategy ladder.

revenue report its shape (Figure 3.16) on first load.

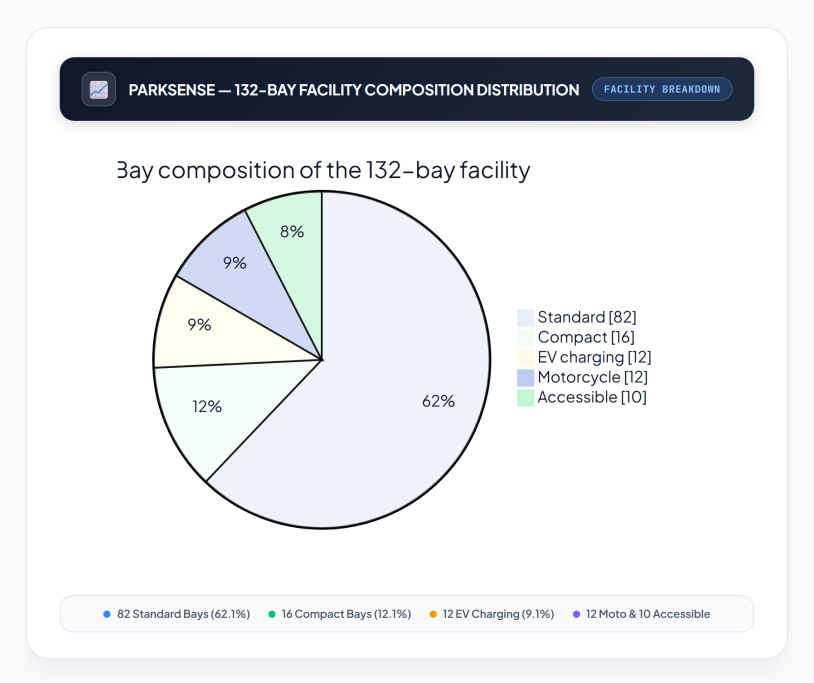

Figure 3.15: Bay-type distribution of the seeded facility.

Figure 3.16: Seeded seven-day revenue trend shown by the reports view.

Chapter 4

Implementation

4.1 Front-End / Interface Design

The control room is a dependency-free single-page application of fifteen static files served by the same process as the API: one shell document, one hand-written stylesheet and thirteen JavaScript modules (an API client, UI helpers, a polling store, a router and nine view modules).

Design system. A dark "control room" identity: deep navy panels with cyan accents, monospace codes for plates, tickets and timestamps, LED-board cards with glowing green/red text, CSS-animated barrier arms, and a bay grid whose cells are coloured by state (green free, amber reserved, red occupied, hatched out-of-service) with type-glyph badges for EV, accessible, compact and motorcycle bays.

Routing. A custom hash router serves nine routes (login, dashboard, map, gates, tickets, members, tariffs, reports, patterns); unauthenticated visitors are redirected to the login route and the navigation bar reflects the active view.

State management. A single `Store` polls the cheap summary, dashboard and event endpoints every three seconds and notifies subscribed views; views unsubscribe on route change, so polling cost scales with the visible page only.

Server state. Poll intervals are short (3 s) because every polled endpoint is an in-memory read; transient failures are swallowed and retried on the next tick so a slow request never blanks the dashboard.

API layer. A `fetch` wrapper attaches the bearer token, parses JSON, translates HTTP statuses into thrown error objects and auto-signs-out on 401 — one error path for all views.

UI techniques. Charts are pure CSS bar columns (no chart library); toasts, modals and the receipt preview are generated by a tiny DOM builder; CSV export simply navigates to the API's CSV format.

4.2 Back-End

The boot class does nothing but start Spring; a single `@Configuration` wires the entire object graph — lot tree, ledger, publisher, boards, stores, selector, hardware factory, gates, proxies, mediator and facade — as ordinary bean methods. Because the domain is annotation-free, every pattern unit-tests without a container; the `@SpringBootTest` flow test exists only to prove the HTTP surface end to end.

Middleware. One servlet filter authenticates every `/api` request (except login and health) and publishes the caller to a thread-local role context; one `@RestControllerAdvice` translates domain exceptions (illegal argument, illegal transition, security) into 400/409/403 responses.

Protection proxy at the object boundary. The interesting security design is not at the HTTP edge: emergency barrier force-open and force-close are guarded by `GateControlProxy`, which wraps the real controller, consults the thread-local role, refuses operators with an audited exception and passes administrators through — so even future code that reuses the controller cannot bypass the check.

Route modules. Nine controllers expose 36 endpoints (auth 3, map 3, gates 4, control 3, tickets 6, members 3, tariffs 4, reports 2, system 8); selected endpoints are listed in Appendix A.2.

Store models. All stores are thread-safe in-memory maps with small query helpers; the tariff store additionally enforces "at most one active plan per kind", and the ticket store derives its open/history views from ticket state rather than duplicate flags.

Persistence bridge. `MongoSync` subscribes to tiny change hooks on the five stores and flushes debounced collection snapshots to the `parksense` Atlas database every few seconds; on boot it replays tickets through the legal state-machine transitions, restores open stays onto their bays via the ledger, re-applies out-of-service marks and resumes the ticket-number sequence — so a restarted process is indistinguishable from a never-stopped one. Unreachable database simply means in-memory mode with a logged warning.

Security. PBKDF2-HMAC-SHA256 (120k iterations, per-user salt) password hashing; opaque random bearer tokens invalidated on logout; ADMIN-only routes for tariff writes, voids, audit, clock and reset; role-refusal reasons never leak internal class names.

4.3 The 16 Design Patterns as Implemented

Table 4.1 maps every GoF pattern the system uses to its concrete classes and the user flow that exercises it — the same catalogue the running system serves at `GET /api/system/patterns` and renders on its Patterns page.

Table 4.1: The 16 GoF patterns in ParkSense.

Pattern	Category	Realisation and exercising flow
Singleton	Creational	`OccupancyLedger` — one truth for slot states; every entry, sensor confirm and exit writes it
Factory Method	Creational	`GateHardwareFactory` with Simulated / Vendor families — all four gates boot through one line-swappable factory

Continued on next page...

Table 4.1 – Continued from previous page

Pattern	Category	Realisation and exercising flow
Builder	Creational	`TariffPlanBuilder` — rejects cap-below-base, bad grace and windowless early-birds before construction
Adapter	Structural	`PlateSenseAdapter`, `ParktronSensorAdapter`, `ParktronBarrierAdapter` — vendor camera frames, bitmask words and motor commands behind internal ports
Decorator	Structural	`CarWashAddon`, `ValetServiceAddon`, `EvChargeAddon`, `LostTicketPenalty` wrapping `BaseParkingFee` — checkout toggles stack them into the receipt
Facade	Structural	`OperationsFacade` — one method per UI action; controllers are single lines
Proxy	Structural	`GateControlProxy` — operator force-open refused (403 + audit), admin passes
Composite	Structural	`ParkingLot` → `Floor` → `Zone` → `Slot` — rollups, the live map and visitors walk one interface
Mediator	Behavioural	`ControlRoomMediator` coordinating entry gates, exit gates, kiosk, boards and sensor feed — colleagues never reference each other
Observer	Behavioural	`OccupancyEventPublisher` — every ledger transition fans out to LED boards, dashboard feed and the LOT-FULL watcher
Strategy	Behavioural	Five `TariffStrategy` implementations behind `TariffSelector` — pricing swaps per ticket and per activation
Command	Behavioural	`GateCommandQueue` with open/close/print/signal commands — executed, audited, undoable from the simulator panel
Template Method	Behavioural	`ExitProcessor` pipeline (fetch → verify → price → settle → free → open) with staffed, express and lost-lane hooks
Chain of Responsibility	Behavioural	Six ordered `EntryRuleHandler`s — blacklist, duplicate, member, availability, EV, capacity; first failure shown verbatim
State	Behavioural	`TicketState` classes ISSUED → ACTIVE → PAID → EXITED (+ LOST, VOID) with overstay reopening
Visitor	Behavioural	`OccupancyVisitor`, `UtilizationVisitor`, `RevenueVisitor`, `PeakHourVisitor`, `AddonsVisitor` — reports walk the domain without touching it

4.4 Modules and Features

The following 13 functional modules were implemented:

1. **Authentication & RBAC** — PBKDF2 logins, bearer sessions, thread-local roles,

ADMIN/OPERATOR separation.

2. **Live Parking Map** — all 132 bays rendered by state and type with filters, click-through bay detail and three-second refreshes.
3. **Gate Simulator (entry)** — camera-read button through the hardware adapter, chain verdict trace, animated barrier and per-gate command history.
4. **Exit Lanes** — staffed booth with cash/card/mobile settlement and change, express member lane at zero charge, and the lost-ticket desk with flat penalty pricing.
5. **Ticketing & Checkout** — open-ticket browser, add-on toggles, pay modal with change preview, printable receipt and grace countdown.
6. **Ticket Lifecycle** — the full state machine including overstay reopening and admin voids that free bays safely.
7. **Members Registry** — pass holders with multi-plate support, expiry visibility and registry editing (ADMIN).
8. **Tariff Administration** — builder-validated plan creation with inline invariant errors and live activation toggles.
9. **Reports & Analytics** — revenue by day, occupancy by zone, utilisation by floor, peak exit hours and add-on uptake, each with chart and CSV export.
10. **Control-Room Dashboard** — KPI tiles, entrance LED board, level boards, capacity alerts and the live occupancy feed.
11. **Audit Trail** — append-only record of every command, proxy denial, payment, void and override (ADMIN view).
12. **System Control** — deterministic reseed, simulated clock advance for overstay demos, health endpoint.
13. **Pattern Catalogue** — the self-documenting guide to all 16 patterns served from the code's own registry.

Chapter 5

User Manual

5.1 System Requirements

5.1.1 Hardware Requirements

Table 5.1: Hardware requirements.

Component	Minimum	Recommended
Processor	1 GHz dual-core	2 GHz quad-core or better
RAM	2 GB (server + browser)	8 GB
Storage	300 MB free (JDK + project)	1 GB SSD
Display	1366×768	1920×1080 (map view benefits)
Network	Loopback only (localhost)	LAN for multi-operator demos
Device	Any PC/laptop	Desktop with a wide screen

5.1.2 Software Requirements

For users. Any modern operating system; a current Chrome, Edge or Firefox; JavaScript enabled. Because the SPA and API are served from one process, only `localhost:8080` is needed.

For local development. Windows 10/11 or any OS; JDK 17 or newer (the project was verified on JDK 23); no database, no environment variables; the Maven wrapper (`mvnw.cmd`) downloads Maven itself; Visual Studio Code with the Java extension pack; a headless-browser QA script (optional) for automated page walkthroughs.

5.2 User Interfaces

5.2.1 Panel A — Signing In

The login card is deliberately self-documenting: the two demo accounts are printed on it (Figure 5.1). Sessions persist across refreshes until sign-out or a server restart.

5.2.2 Panel B — Operations Workspace

After sign-in the operator lands on the control-room dashboard (Figure 5.2): KPI tiles, the entrance LED board with per-level summaries, the capacity alert list and the live occupancy feed. The live map (Figure 5.3) shows every bay of the three floors coloured by state with a side Bay Inspector that reveals the sensor node, plate, ticket and occupancy time of any selected bay, and the gate simulator (Figure 5.4) drives entries and exits with full rule traces, command history and undo.

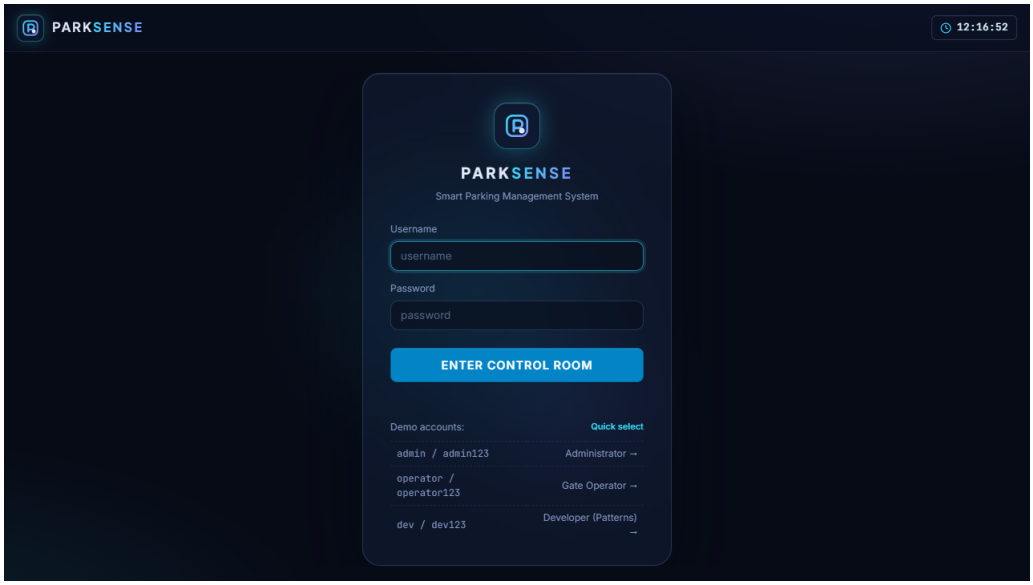

Figure 5.1: Sign-in card with demo credentials.

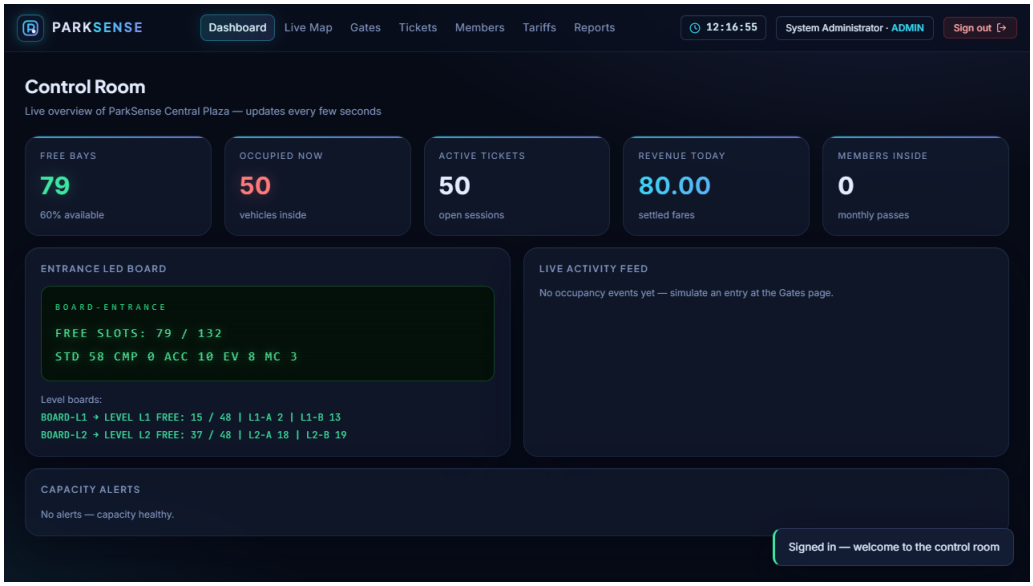

Figure 5.2: Control-room dashboard: KPIs, LED board, alerts, live feed.

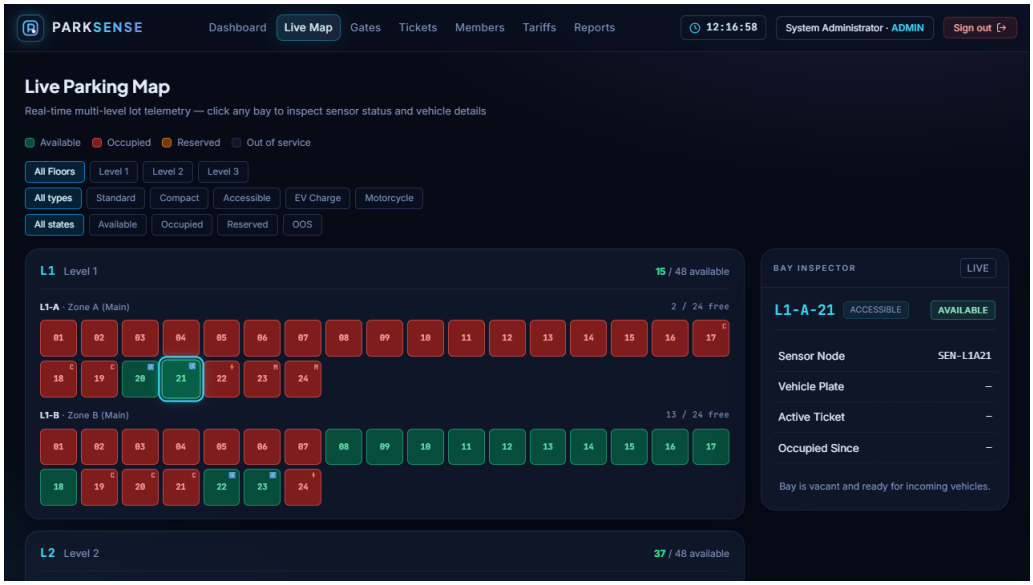

Figure 5.3: Live parking map with bay states and the Bay Inspector panel.

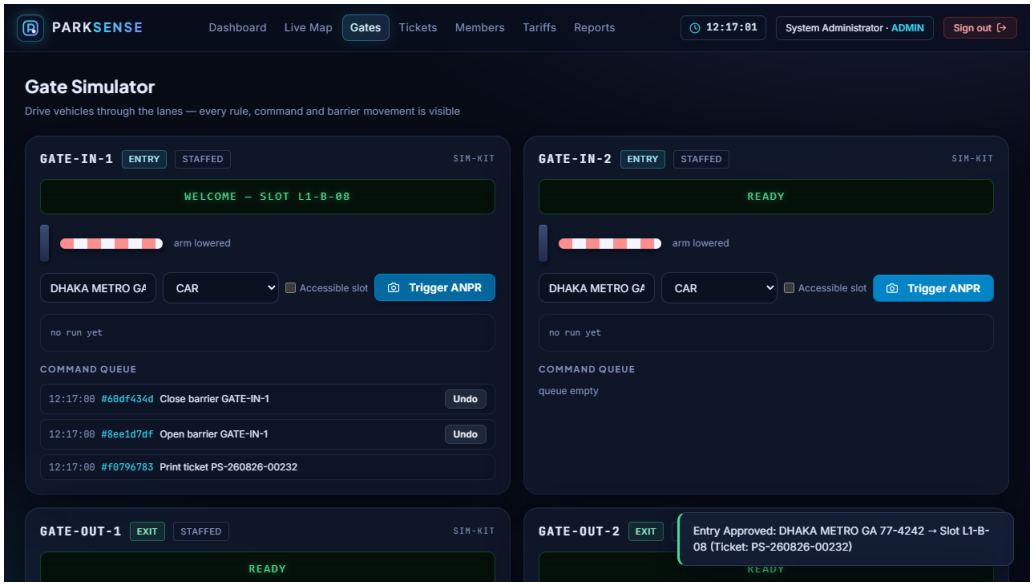

Figure 5.4: Gate simulator: entry chain verdict, barrier, command queue.

5.2.3 Panel C — Checkout and Administration

The checkout view (Figure 5.5) settles fares: add-on toggles stack fee lines, the pay modal computes change, and the receipt (Figure 5.6) prints the breakdown with the grace deadline. Administration covers the member registry (Figure 5.7), the tariff plan cards with builder-validated creation (Figure 5.8), the reports workspace with charts and CSV export (Figure 5.9) and the pattern catalogue (Figure 5.10).

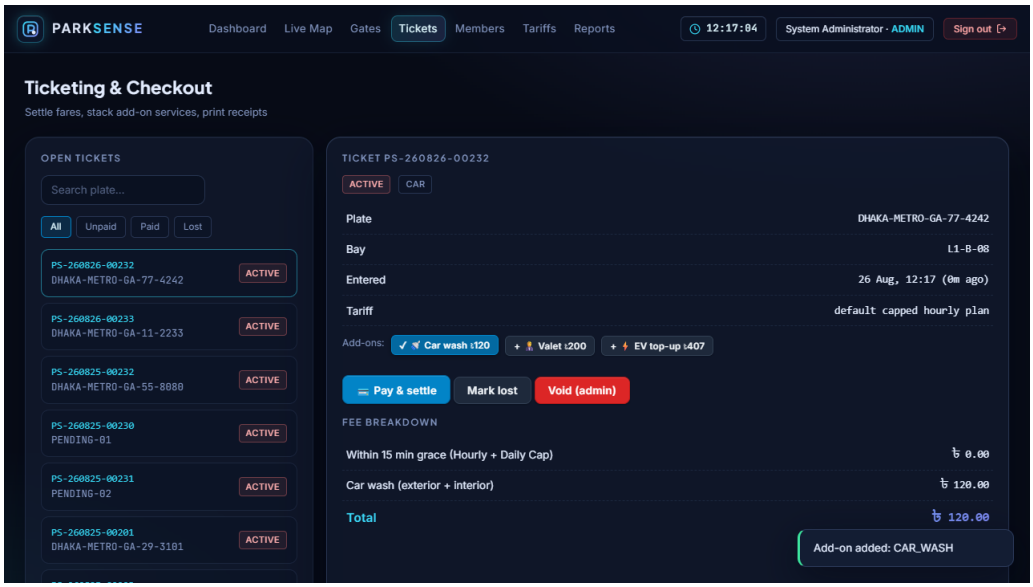

Figure 5.5: Checkout: fee breakdown with stacked add-on decorators.

Figure 5.6: Payment receipt with grace-window deadline.

5.2.4 Login Credentials

Administrator Access: full control including barrier override, tariff editing and the audit trail

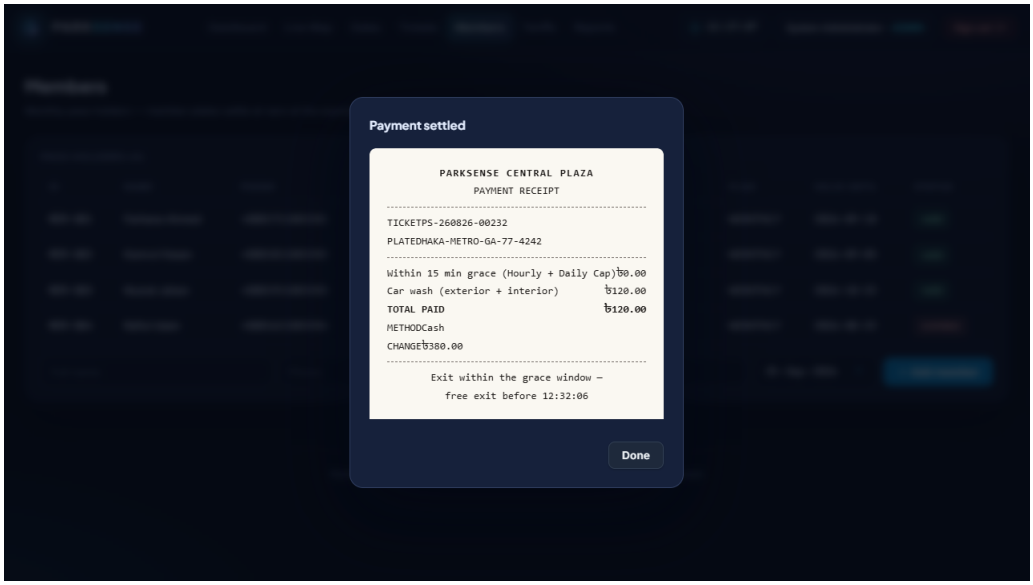

Figure 5.7: Member registry with pass validity.

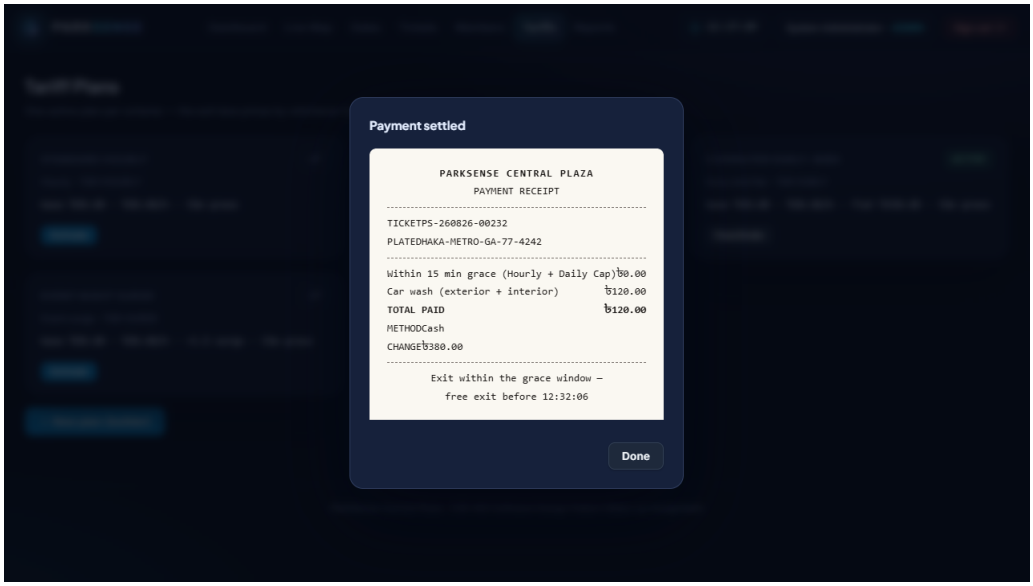

Figure 5.8: Tariff administration: plan cards and activation.

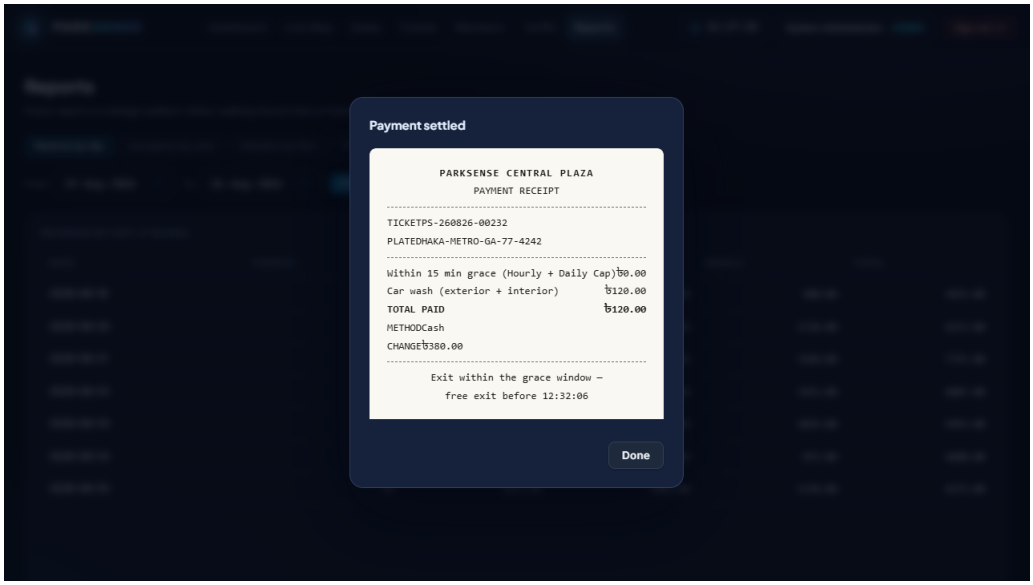

Figure 5.9: Reports workspace: revenue chart and tables.

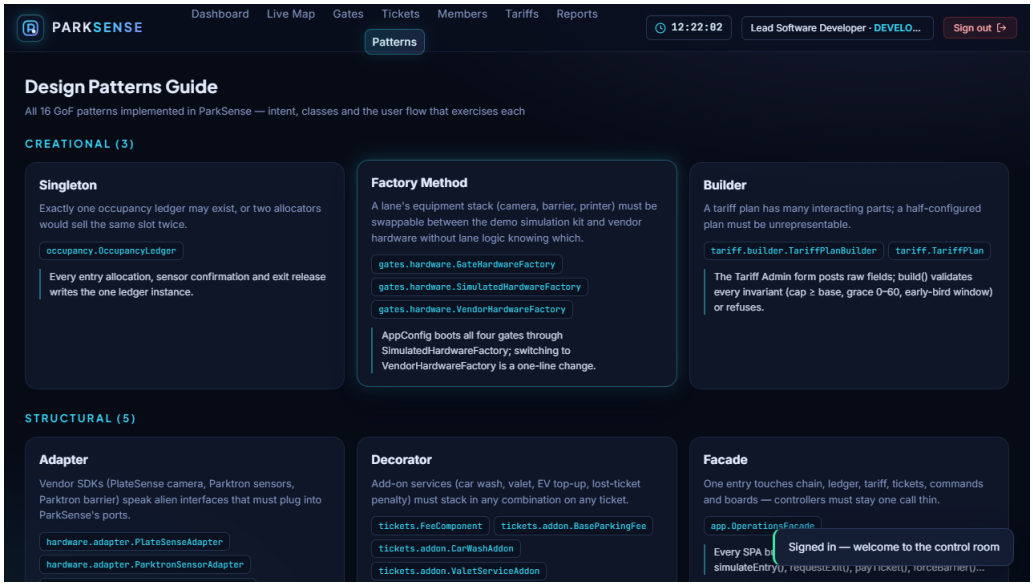

Figure 5.10: Design-pattern catalogue rendered from the live system.

- **Username:** `admin`
- **Password:** `admin123`

Operator Access: lanes, tickets, checkout and live views

- **Username:** `operator`
- **Password:** `operator123`

Developer Access: everything above plus the live design-pattern catalogue (the Patterns tab is visible only to this role)

- **Username:** `dev`
- **Password:** `dev123`

5.3 Getting Started

1. Install JDK 17+ and unzip the project.
2. In the project folder run `.\mvnw.cmd spring-boot:run` (first run downloads dependencies).
3. Open `http://localhost:8080` and sign in as `admin/admin123`.
4. The facility boots seeded — drive a car in from the Gates page, settle it at Tickets, and let it out again.
5. To verify the build, run `.\mvnw.cmd clean test` — 91 tests across 15 classes.

Chapter 6

Conclusion

6.1 Conclusion

ParkSense matured into a complete, self-contained smart-parking facility: 132 bays across three floors, four gates, seven in-memory stores, nine REST controllers with 36 endpoints, 13 functional modules and — the project's purpose — **16 GoF design patterns**, each load-bearing in a real flow rather than decorative. The domain stayed framework-free plain Java throughout, which is precisely why 91 automated tests could verify every pattern in isolation and a full-stack test could walk a car from camera read to freed bay through the real HTTP surface. The security posture is honest for a laboratory: hashed credentials, token sessions, three roles enforced at both the edge and the object boundary (the DEVELOPER role additionally unlocks the live pattern catalogue), and an append-only audit trail that records refusals as faithfully as approvals.

6.2 Limitations

1. **Persistence is optional.** With `MONGODB_URI` configured the `parksense` Atlas database keeps everything across restarts; without it the zero-setup in-memory fallback reseeds on every start, accumulating no real history.
2. **Simulated hardware.** The ANPR camera, sensor network and barrier motors are in-process stand-ins; the adapter seams are ready, but no physical device has been integrated.
3. **No external payments.** Cash, card and mobile are recorded settlements, not gateway integrations.
4. **Single node.** One process serves everything; there is no clustering, no horizontal scaling, no offline kiosk mode.
5. **No driver-facing app.** Bays are guided on in-facility boards and the staff console only; drivers receive no pre-arrival availability or navigation.
6. **Poll-based live view.** The SPA polls every three seconds instead of using web-sockets, so updates lag up to one interval.
7. **Simulated clock dependency.** Overstay and expiry demos use the clock-advance control; real deployments would need NTP-driven time and day-boundary policies.

6.3 Future Works

The natural next steps follow the limitation list. First, persistence: a document database behind the existing store interfaces, with the seeded determinism retained as a demo profile. Second, real hardware through the already-isolated adapter and factory seams — a live PlateSense camera and Parktron nodes would exercise exactly the code paths the stand-ins occupy. Third, a driver mobile application for pre-arrival

availability, bay reservation and contactless payment at the exit lane. Fourth, push-based live updates (websockets or server-sent events) replacing polling, making the barrier animation event-accurate. Fifth, demand-responsive pricing: the strategy set is designed for extension, and a forecasting strategy could learn hourly multipliers from the ticket history the visitors already walk. Finally, multi-site federation — one mediator per facility behind a shared tariff and member registry — is a straightforward composition of the current architecture.

References

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[2] E. Freeman and E. Robson, *Head First Design Patterns*, 2nd ed. O'Reilly Media, 2021.

[3] Refactoring.Guru. "Design Patterns." `https://refactoring.guru/design-patterns`, 2026.

[4] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[5] I. Y. Yusof, Z. Ibrahim, and S. A. Mostafa, "A survey of smart parking systems: architecture, technologies and open issues," *Internet of Things*, 2023.

[6] Oracle. "Java SE 17 Documentation." `https://docs.oracle.com/en/java/javase/17/`, 2026.

[7] VMware. "Spring Boot Reference Documentation." `https://docs.spring.io/spring-boot/index.html`, 2026.

[8] VMware. "Spring Web MVC Documentation." `https://docs.spring.io/spring-framework/reference/web/webmvc.html`, 2026.

[9] FasterXML. "Jackson Project Wiki." `https://github.com/FasterXML/jackson`, 2026.

[10] MongoDB, Inc. "MongoDB Java Driver Documentation." `https://www.mongodb.com/docs/drivers/java/sync/current/`, 2026.

[11] JUnit Team. "JUnit 5 User Guide." `https://junit.org/junit5/docs/current/user-guide/`, 2026.

[12] Apache Software Foundation. "Apache Maven Documentation." `https://maven.apache.org/guides/`, 2026.

[13] Apache Software Foundation. "Apache Tomcat 10 Documentation." `https://tomcat.apache.org/tomcat-10.1-doc/`, 2026.

[14] B. Kaliski, "PKCS #5: Password-Based Cryptography Specification Version 2.0," RFC 2898, IETF, 2000.

[15] OWASP Foundation. "Password Storage Cheat Sheet." `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`, 2026.

[16] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[17] Mozilla. "MDN Web Docs: Fetch API, CSS Grid, JavaScript." `https://developer.mozilla.org/`, 2026.

[18] Mozilla. "MDN Web Docs: Window: hashchange event." https://developer.mozilla.org/en-US/docs/Web/API/Window/hashchange_event, 2026.

[19] Oracle. "Class BigDecimal (Java SE 17 API)." https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html, 2026.

[20] Oracle. "The Java Tutorials: Concurrency." https://docs.oracle.com/javase/tutorial/essential/concurrency/, 2026.

[21] Microsoft. "Visual Studio Code Documentation." https://code.visualstudio.com/docs, 2026.

[22] Postman, Inc. "Postman Learning Center." https://learning.postman.com/, 2026.

[23] Software Freedom Conservancy. "Git Documentation." https://git-scm.com/doc, 2026.

[24] Mermaid. "Mermaid Diagramming and Charting." https://mermaid.js.org/, 2026.

[25] Mermaid. "Mermaid Live Editor." https://mermaid.live, 2026.

[26] W3C. "Web Content Accessibility Guidelines (WCAG) 2.1." https://www.w3.org/WAI/WCAG21/quickref/, 2026.

[27] Lucidchart. "What is a Data Flow Diagram." https://www.lucidchart.com/pages/data-flow-diagram, 2026.

[28] Creately. "Use Case Diagram Tutorial." https://creately.com/guides/use-case-diagram-tutorial/, 2026.

[29] InterviewBit. "System Architecture Tutorial." https://www.interviewbit.com/blog/system-architecture/, 2026.

[30] Visual Paradigm. "What is a State Machine Diagram?" https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-state-machine-diagram/, 2026.

[31] The Daily Star. "Parking mess in Dhaka: how the city loses streets and footpaths to unmanaged parking." https://www.thedailystar.net/, 2025.

Appendix A

Glossary and Supplementary Material

A.1 Glossary of Terms

Table A.1: Glossary of terms.

Term	Meaning
ANPR	Automatic Number Plate Recognition — camera reading of licence plates
Bay / Slot	One parking position; typed (standard, compact, accessible, EV, motorcycle)
GoF	"Gang of Four" — the four authors of the canonical design-pattern catalogue
Ledger	The singleton authority that alone may change bay states
Grace window	Free minutes after payment within which the exit must occur (default 15)
Early bird	Flat-rate plan for arrivals before 10:00 leaving after 16:00
Event surge	Multiplier pricing activated by the operator around events
Member pass	Monthly subscription settling recognised plates at zero
Express lane	Exit lane for pass holders only; staffed lane takes everyone
Lost-ticket penalty	Flat administrative fine plus one day rate when the ticket is missing
Overstay	Leaving after the grace window; the fare reopens for repayment
Mediator	The control-room object coordinating all lane colleagues
Occupancy event	Immutable record of one bay state transition, fanned out to observers
Protection proxy	Object wrapper that refuses non-ADMIN barrier overrides
Facade	The single entry class behind which all subsystems are hidden
RBAC	Role-based access control — ADMIN and OPERATOR here
PBKDF2	Password-based key derivation function used for password hashing
SPA	Single-page application — the whole control room is one page
DFD	Data-flow diagram (level 0 context, level 1 decomposition)

A.2 Key API Endpoints

Table A.2: Key API Endpoints in ParkSense.

Method	Endpoint	Purpose	Auth
POST	`/api/auth/login`	Exchange credentials for a bearer token	Public
GET	`/api/auth/me`	Current session user and role	Token
POST	`/api/auth/logout`	Invalidate the session token	Token
GET	`/api/map`	Full lot tree with bay states	Token
GET	`/api/map/summary`	Cheap per-type/per-floor free counts	Token
GET	`/api/map/slots`	Filtered slot list (floor/type/state)	Token
GET	`/api/gates`	All four gate panels with queues	Token
POST	`/api/gates/{id}/entry`	Camera-read entry (chain + mediator)	Token
POST	`/api/gates/{id}/exit`	Exit through the lane processor	Token
POST	`/api/gates/{id}/commands/{cmd}/undo`	Reverse a gate command	Token
POST	`/api/control/gates/{id}/force-open`	Emergency override (proxy-guarded)	Admin
POST	`/api/control/gates/{id}/force-close`	Emergency close (proxy-guarded)	Admin
GET	`/api/tickets`	Ticket list with state/plate filters	Token
GET	`/api/tickets/{no}`	Ticket detail with fee chain	Token
POST	`/api/tickets/{no}/addons`	Toggle an add-on decorator	Token
POST	`/api/tickets/{no}/pay`	Settle with method + tendered	Token
POST	`/api/tickets/{no}/lost`	Mark ticket lost (penalty tariff)	Token
POST	`/api/tickets/{no}/void`	Cancel ticket and free the bay	Admin
GET	`/api/members`	Pass-holder registry	Token
POST	`/api/members`	Add a member	Admin
GET	`/api/tariffs`	Plan cards with activation state	Token
POST	`/api/tariffs`	Builder-validated plan creation	Admin
POST	`/api/tariffs/{id}/activate`	Activate a plan (deactivates sibling)	Admin
GET	`/api/reports/{name}`	Revenue / occupancy / utilization / peak-hours / addons (JSON or CSV)	Token
GET	`/api/reports/dashboard`	KPI tile values	Token
GET	`/api/system/boards`	LED-board snapshots	Token
GET	`/api/system/events`	Live occupancy feed	Token
GET	`/api/system/patterns`	The 16-pattern catalogue	Token
GET	`/api/system/audit`	Append-only audit trail	Admin
POST	`/api/system/reset`	Reseed the facility	Admin
POST	`/api/system/clock`	Advance simulated time (overstay demos)	Admin

A.3 Project Timeline

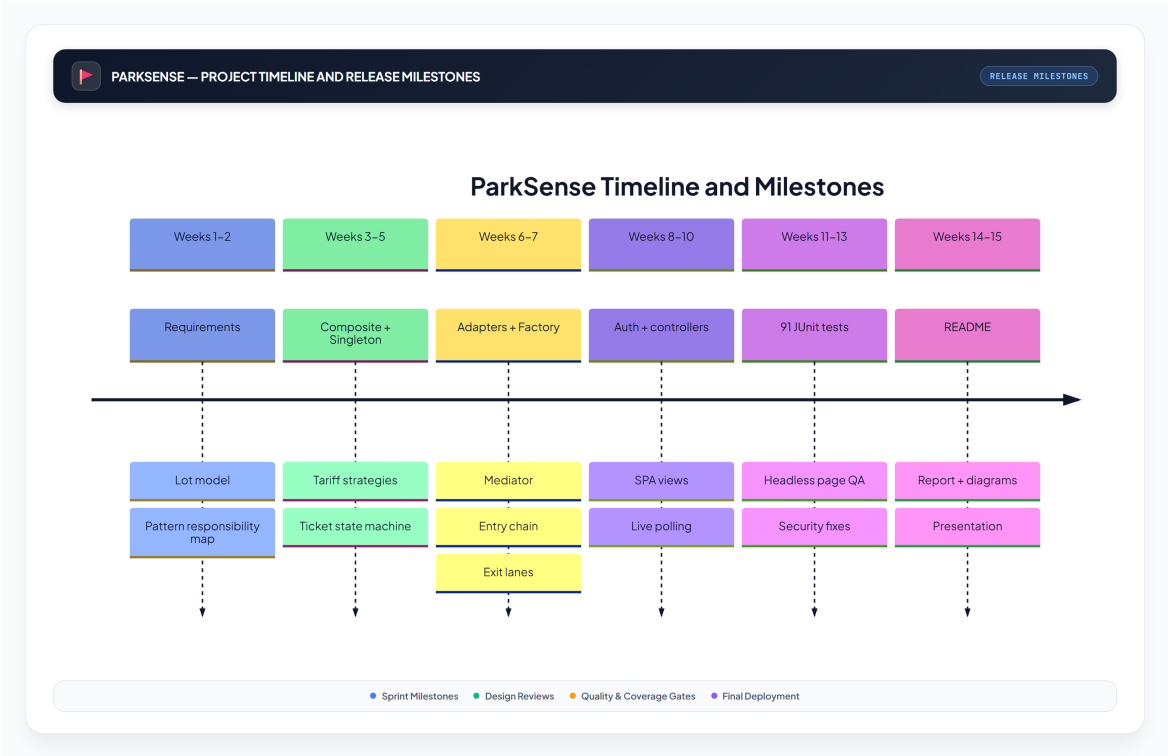

Figure A.1: Project Timeline and Release Milestones.

— End of Document —